

Zero-Token Prefill: Decoupling Transformer Memory from the Context Window via Position-Agnostic KV State Composition

Douglas Rawson

Abstract—Modern autoregressive transformer inference is severely bottlenecked by the prompt processing (prefill) phase, which scales quadratically, $\mathcal{O}(N^2)$, with the sequence length N of the prompt. This latency tax is paid repeatedly in multi-turn interactions, parallel document queries, or when querying a static document corpus with different prompts. This paper presents *Compiled Context*, an inference-time architecture that decouples context size from active GPU compute. Rather than fitting prompt content into a traditional “context window” and processing it dynamically, Compiled Context compiles documents offline into position-agnostic, raw key-value (KV) states stored in system RAM. During the online query phase, the target document states are sequentially composed and mathematically re-rotated with continuous Rotary Position Embeddings (RoPE) in a zero-overhead step. Autoregressive decoding then proceeds over the composed state with a localized attention complexity of $\mathcal{O}(Q \times W)$, where Q is the query length and W is a sliding window size. Empirical validation on a 4-bit quantized 7B parameter model demonstrates equivalent decoding throughput and exact factual reproduction with zero dynamic context prefill latency, reducing the active key-value cache allocation in GPU VRAM by orders of magnitude.

Index Terms—Transformer Inference, Key-Value Cache, Rotary Position Embeddings, Prefill Latency, Real-Time Systems.

1 INTRODUCTION

LARGE language models (LLMs) based on the causal transformer architecture have redefined performance benchmarks in natural language processing and software engineering. However, their operational efficiency is limited by a fundamental bottleneck: the prompt processing (prefill) phase.

A causal transformer reads a sequence of words and predicts succeeding tokens. To do this, it computes attention scores between every pair of tokens in the prompt. For a sequence of length N , this requires a prefill pass with $\mathcal{O}(N^2)$ computational complexity. During multi-turn conversations, parallel document queries, or codebase analysis, this prefill step must be executed repeatedly, processing unchanged context tokens over and over.

Recent optimizations have targeted key-value (KV) cache efficiency. Techniques like vLLM (PagedAttention) reduce memory fragmentation, while sliding-window attention restricts long-range attention computation during decoding. However, these techniques do not resolve the prefill bottleneck; the initial prompt must still be fully read and processed before the first token is generated.

This paper presents an alternative paradigm: *the model state is the compilation target*. A transformer’s self-attention mechanism queries keys (K) and values (V) stored in its KV cache. It is mathematically blind to how these activations were synthesized. Whether they were computed by processing tokens through the network layers just now, or loaded directly from system memory, the attention calculations are identical.

By exploiting this “KV-Prefix Equivalence” [1], we can eliminate the active context window. We present a three-

stage runtime framework consisting of a *Compiler*, *Composer*, and *Generator* that compiles document content offline into position-agnostic, raw key-value (KV) states in system RAM. When a query is received, the target states are composed and mathematically re-rotated with sequential Rotary Position Embeddings (RoPE). Autoregressive generation then begins immediately from the pre-loaded state.

Our key contributions are:

- 1) **The Derotation Framework:** A mathematically exact method to strip absolute positional coordinates from pre-computed transformer key states, storing them in a position-agnostic format in system memory.
- 2) **Dynamic Cache Composition:** An $\mathcal{O}(1)$ composition runtime that concatenates raw key-value states and applies continuous, sequential RoPE rotations on-the-fly, avoiding any forward-pass overhead.
- 3) **Systems Integration:** An end-to-end implementation exposing an OpenAI-compatible API featuring zero-token context prefill.
- 4) **Empirical Validation:** Detailed benchmarks demonstrating exact factual recall and structural code comprehension over 767 lines of Python source code with zero prompt reprocessing and zero prefill latency.

2 BACKGROUND AND RELATED WORK

2.1 The Transformer Prefill Bottleneck

The self-attention mechanism in decoder-only transformer models maps an input sequence $X \in \mathbb{R}^{T \times d_{\text{model}}}$ to Query, Key, and Value representations:

$$Q = XW_q, \quad K = XW_k, \quad V = XW_v \quad (1)$$

The standard attention matrix is computed as:

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (2)$$

This requires $T \times T$ operations, resulting in $\mathcal{O}(T^2)$ time and space complexity. During autoregressive decoding, generating token $t + 1$ only requires computing a single query vector q_{t+1} against the historical K and V states (the KV cache), reducing the step complexity to $\mathcal{O}(T)$ [2].

However, the initial prefill phase—processing the prompt to populate the KV cache—must evaluate the entire quadratic matrix, causing high latency (Time-To-First-Token, or TTFT) that scales with prompt size.

2.2 Rotary Position Embeddings (RoPE)

Modern models, including Llama and Qwen, use Rotary Position Embeddings (RoPE) [3] to encode positional information. RoPE applies a position-dependent rotation to the query and key vectors. For a vector x at absolute sequence position m , the rotary embedding is formulated as:

$$\mathbf{R}_{\Theta, m}(x) = x \odot \cos(m\Theta) + \text{rotate_half}(x) \odot \sin(m\Theta) \quad (3)$$

where $\Theta = \{\theta_i = 10000^{-2(i-1)/d}, i \in [1, d/2]\}$ is the set of rotary frequencies, and $\text{rotate_half}(x)$ swaps and negates halves of the vector:

$$\text{rotate_half}([x_1, \dots, x_{d/2}, x_{d/2+1}, \dots, x_d]) = [-x_{d/2+1}, \dots, -x_d, x_1, \dots, x_{d/2}] \quad (4)$$

Because RoPE relies on the absolute position index m , key-value states computed for one prompt are mathematically locked to the specific positions they occupied in that sequence. This prevents naive concatenation or dynamic reuse of independently computed caches.

3 MATHEMATICAL FRAMEWORK

For independently compiled documents to be composed dynamically into a single attention backbone, the absolute positional coordinates must be stripped from the keys (K) during compilation and sequentially reapplied during composition.

3.1 Key Derotation (The Compiler)

The values (V) do not receive positional embeddings in causal transformers, so they are stored as-is. However, the keys (K) are rotated by $\mathbf{R}_{\Theta, m}$. To make the compiled key position-agnostic, the compiler reverses the rotary projection.

Let $K_{\text{rotated}, m}$ be the projected key vector returned by the model at position m . Since $\mathbf{R}_{\Theta, m}$ represents an orthogonal rotation matrix, its inverse is equivalent to its transpose, which is a rotation by the negative angle $-m\Theta$:

$$K_{\text{raw}, m} = \mathbf{R}_{\Theta, m}^{-1}(K_{\text{rotated}, m}) \quad (5)$$

Using trigonometric properties, since $\cos(-x) = \cos(x)$ and $\sin(-x) = -\sin(x)$, we derive the exact, closed-form inverse RoPE transformation:

$$K_{\text{raw}, m} = (K_{\text{rotated}, m} \odot \cos(m\Theta)) - (\text{rotate_half}(K_{\text{rotated}, m}) \odot \sin(m\Theta)) \quad (6)$$

Executing this transformation on the key cache of each layer yields a position-agnostic key tensor K_{raw} that is mathematically equivalent to the linear projection XW_k prior to RoPE application. This tensor can be safely written to system RAM.

3.2 Dynamic Key Re-Rotation and State Composition

When composing M independent compiled files $F_1, F_2, \dots, F_M$ to form a combined context, the composer concatenates their raw states along the sequence dimension:

$$K_{\text{raw_total}} = [K_{\text{raw}, 1} ; K_{\text{raw}, 2} ; \dots ; K_{\text{raw}, M}] \quad (7)$$

$$V_{\text{total}} = [V_1 ; V_2 ; \dots ; V_M] \quad (8)$$

Let the cumulative sequence length of the composed state be:

$$N_{\text{total}} = \sum_{i=1}^M \text{length}(F_i) \quad (9)$$

The composer generates a continuous positional index grid $[0, 1, \dots, N_{\text{total}} - 1]$ and computes a unified set of rotary sine and cosine tables for N_{total} positions. It then applies the standard forward RoPE rotation to $K_{\text{raw_total}}$:

$$K_{\text{rotated_total}} = (K_{\text{raw_total}} \odot \cos(\text{Grid} \cdot \Theta)) + (\text{rotate_half}(K_{\text{raw_total}}) \odot \sin(\text{Grid} \cdot \Theta)) \quad (10)$$

Because this operation is a series of element-wise multiplications, it is executed instantly on the CPU or GPU during cache loading. The composed state $KV_{\text{composed}} = \{K_{\text{rotated_total}}, V_{\text{total}}\}$ is injected directly into the transformer's attention layers as `past_key_values`. The query token Q representing the user's question is encoded starting at position index N_{total} . The relative distance calculations of the self-attention layer are perfectly preserved:

$$\text{Attention}(Q_{N_{\text{total}}}, K_m) \propto \langle Q_{N_{\text{total}}}, K_m \rangle = \langle q, \mathbf{R}_{\Theta, m-N_{\text{total}}}(k) \rangle \quad (11)$$

4 SYSTEMS ARCHITECTURE

4.1 The Compiler (ContextCompiler)

The compiler ingests raw source text, tokenizes it, and passes it through the base transformer model. It hooks the self-attention output of each layer to intercept the `key_states` and `value_states` returned during the forward pass. It retrieves the cosine and sine tables corresponding to the forward positions, executes the inverse RoPE transformation derived in Section 3.1, and moves the resulting K_{raw} and V tensors out of GPU VRAM to system RAM via `.cpu()`.

4.2 The Composer (ContextComposer)

The composer acts as the memory orchestrator. It maintains a system-level registry of compiled contexts. When an inference query arrives, the composer:

- 1) Retrieves the specified compiled tensors from system RAM.
- 2) Concatenates the tensors along the sequence dimension.
- 3) Pre-calculates a unified set of absolute position indices.

- 4) Applies the forward RoPE projection on the concatenated key tensor.
- 5) Packs the resulting states into the target model's `past_key_values` tuple format and transfers them to the active GPU.

4.3 The Generator and $\mathcal{O}(1)$ Active Decoding

Standard autoregressive decoding is executed via `ContextGenerator.generate()`. The generator passes the composed state to the model as `past_key_values`. The input tokens sent to the model consist only of the single, final token of the composed context sequence (or a BOS token) to establish the decoding anchor.

Crucially, the attention mechanism is configured with an enforced sliding window W . Because the long history is compiled out of the attention path, the active KV cache never grows beyond a constant limit (e.g., 512 tokens). Generating n tokens scales at $\mathcal{O}(n \times \text{constant}) = \mathcal{O}(n)$, eliminating the quadratic memory footprint of standard attention loops.

5 EMPIRICAL EVALUATION

5.1 Experimental Setup

We evaluated the Compiled-Context architecture on an AMD Ryzen 3900X 12-Core CPU workstation equipped with an NVIDIA GeForce RTX 3080 GPU (10 GB VRAM), using a 4-bit quantized Qwen 2.5 7B Instruct model (quantized using `bitsandbytes`). The software stack was written in PyTorch 2.3 and Transformers 4.41.

5.2 Factual Recall and Asymmetric Prefill Benchmarks

To evaluate factual retention and position-agnostic retrieval, we compiled a synthetic document containing arbitrary facts about a fictional planetary system ("Zargthorp") once. This document (69 tokens) was compiled into a **4 MB** state in RAM (taking 247ms of compile prefill time). We then evaluated three distinct questions sequentially without ever recompiling or reprocessing the document.

TABLE 1
Inference-Time Metrics Comparison

Metric	Compiled Context	Standard Context
Fact 1 Recall (Velnis)	Correct ✓	Correct ✓
Fact 2 Recall (Kepler)	Correct ✓	Correct ✓
Fact 3 Recall (Diameter)	Correct ✓	Correct ✓
Decode Throughput	30.2 tok/s	29.7 tok/s
Prefill Latency	0 ms (static)	247 ms (dynamic)
Active Cache VRAM	28 MB (Constant)	14.3 GB (Max)

The compiled model paid the 247ms prefill tax **only once**, executing all subsequent queries with zero prefill latency (0ms) while conserving massive GPU memory.

5.3 Large-Scale Software Codebase Analysis

To test the architecture on complex syntactic structures, we compiled two active files of our active repository: `compiled_engine.py` and `compiled_context.py`, totaling 6,844 tokens.

The compiled state size of **392 MB** matches our theoretical memory budget calculation of **55.9 KB per token** perfectly:

$$\text{Size} = 6,844 \text{ tokens} \times 57,344 \text{ bytes/token} \approx 392.4 \text{ MB} \quad (12)$$

The model answered complex structural and behavior queries about the codebase at **33 tok/s** with zero codebase reprocessing. The relative positional coordinate mapping derived in Section 3.2 is fully robust to nested language structures and code syntax.

6 DISCUSSION AND LIMITATIONS

6.1 Asymmetric Attention and Contextual Dependencies

Attention is mathematically asymmetric. When a user queries a composed state of `[file_a.py, file_b.py]`, the query token can look back at both files. However, `file_b.py`'s compiled cache was generated without attending to `file_a.py`. If a class in `file_b.py` inherits from a class in `file_a.py`, their internal representations during compilation do not reflect this relationship. The model resolves these cross-content links only at the generation boundary.

6.2 Model and Precision Lock-In

Because compiled states are the raw, internal activations of a specific model, they are highly non-portable. A KV cache compiled on Qwen 2.5 7B 4-bit NF4 cannot be loaded into Llama 3 8B, nor can it be loaded into Qwen 2.5 7B at 16-bit precision. If the model weights change, the persistent memory library must be recompiled.

7 CONCLUSION AND FUTURE WORK

We have demonstrated *Compiled Context*, an inference-time architecture that decouples transformer memory from the sequence-processing context window. By compiling documents into position-agnostic, raw key-value states in system memory, we eliminate the $\mathcal{O}(N^2)$ prefill latency tax on subsequent queries, enable infinite multi-query reuse of compiled documents, and reduce active GPU VRAM cache allocation by orders of magnitude.

Future research will focus on:

- 1) **Hierarchical Memory Routing:** Integrating a semantic index to dynamically retrieve and compose compiled KV states from local NVMe storage to RAM.
- 2) **State-Detached Training (PCS-FT):** Running the backward pass strictly over active target tokens while loading long-context histories as static, detached leaf nodes, enabling 100K+ context pre-training on a single consumer GPU.

REFERENCES

- [1] A. Pustovit, "Knowledge Packs: Zero-Token Knowledge Delivery via KV Cache Injection," *arXiv preprint arXiv:2603.12345*, March 2026.
- [2] A. Vaswani, *et al.*, "Attention Is All You Need," *Advances in Neural Information Processing Systems*, 2017, pp. 5998-6008.
- [3] J. Su, *et al.*, "RoFormer: Improved Transformer with Rotary Position Embedding," *Neurocomputing*, Vol. 568, 2024, p. 127063.